



PROJECT REPORT

E-Commerce Sales Analytics

A Python and pandas based command-line application for exploring e-commerce sales performance

Student Name	Bhavya Chauhan
Registration No.	26BAI10579
Project Title	E-Commerce Sales Analytics
Technology	Python 3, pandas
Submission	GitHub repository and project report (PDF)
Date	September 2026

Table of Contents

2. Introduction	3
3. Problem Statement	4
4. Functional Requirements	5
5. Non-functional Requirements	6
6. System Architecture	7
7. Design Diagrams	9
8. Design Decisions and Rationale	14
9. Implementation Details	15
10. Screenshots and Results	17
11. Testing Approach	23
12. Challenges Faced	24
13. Learnings and Key Takeaways	25
14. Future Enhancements	26
15. References	27

2. Introduction

Online retailers generate a continuous stream of order records, but raw rows on their own say little about how the business is doing. Managers want quick answers to questions such as: how much revenue and profit have we made, which category earns the most, which city buys the most, and which payment method do customers prefer?

E-Commerce Sales Analytics is a menu-driven console application written in Python that answers exactly these questions. It loads a set of e-commerce orders into a pandas DataFrame and lets the user view key performance indicators (KPIs), grouped summaries by category, product, city, month and payment method, and browse the raw records. A filter feature lets the user narrow every report to a chosen category and city, and a reset option restores the full data set.

The project demonstrates practical use of data structures, functions, control flow, user input handling and the pandas library (groupby, aggregation, sorting, date handling and Boolean filtering), all in a single, easy-to-run Python file.

3. Problem Statement

Sales data is normally stored as flat tables of orders. Reading such a table row by row is slow and error-prone, and spreadsheets are not always available or convenient for quick summaries. Small teams and students need a lightweight tool that can turn order-level data into clear business insights without complex setup.

The problem: design and build a simple, reliable, interactive program that takes e-commerce order data and produces summary reports (KPIs, category, product, city, monthly and payment-wise analysis) and supports filtering by category and city, while being easy to run, easy to understand and easy to extend.

Objectives

- Load and prepare order data using pandas, including converting order dates to a proper datetime type.
- Compute KPIs: total sales, total profit, number of orders and number of unique customers.
- Provide grouped analyses of sales and profit across categories, products, cities, months and payment methods.
- Allow interactive filtering by category and city and a one-step filter reset.
- Present results in clean, readable console tables with graceful handling of invalid input.

4. Functional Requirements

The functional requirements describe what the system must do. Each is traceable to a menu option in the application.

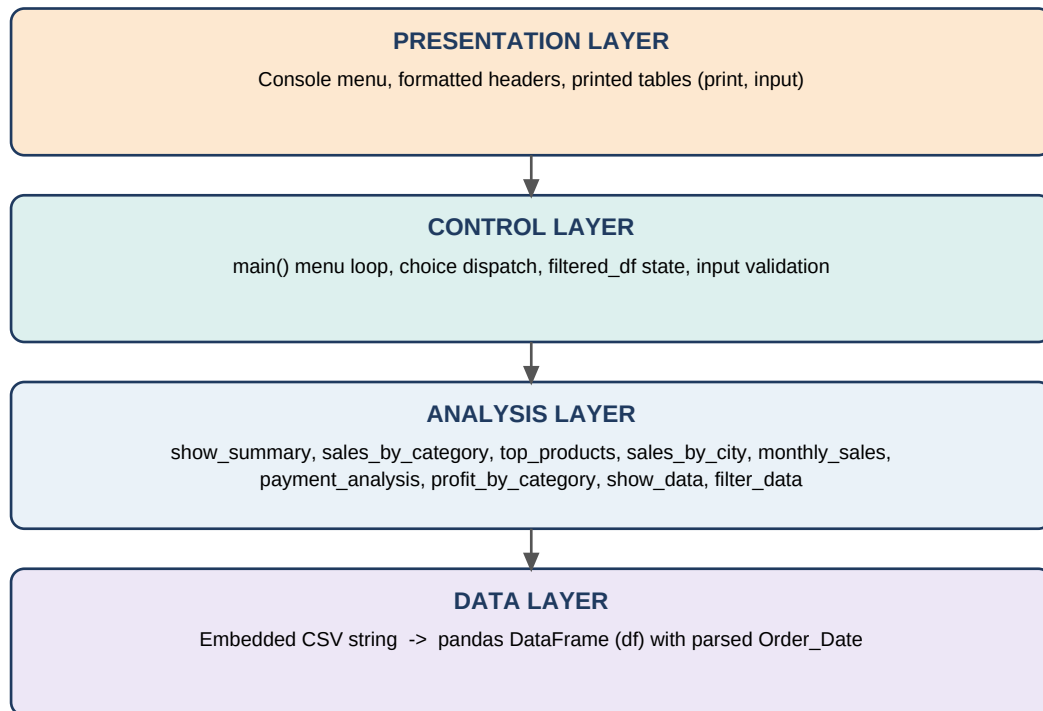
ID	Requirement	Menu option
FR1	The system shall load order data (Order_ID, Order_Date, Customer_ID, Product, Category, City, Sales, Profit, Payment_Method) into a DataFrame and convert Order_Date to datetime at start-up.	Start-up
FR2	The system shall display a main menu and repeat it until the user chooses to exit.	Main menu
FR3	The system shall display KPIs: total sales, total profit, total orders and total unique customers.	1
FR4	The system shall show total sales per category, sorted from highest to lowest.	2
FR5	The system shall show the top 10 products ranked by total sales.	3
FR6	The system shall show total sales per city, sorted from highest to lowest.	4
FR7	The system shall show total sales per calendar month.	5
FR8	The system shall show total sales per payment method, sorted from highest to lowest.	6
FR9	The system shall show total profit per category, sorted from highest to lowest.	7
FR10	The system shall display all records of the currently active data set.	8
FR11	The system shall let the user filter data by category and by city, each with an 'All' option.	9
FR12	The system shall let the user reset all filters to the full data set.	10
FR13	The system shall apply the active filter to every report (options 1 to 8).	1 to 8
FR14	The system shall reject invalid menu choices with a message and show the menu again.	Any
FR15	The system shall exit cleanly and print the project credit.	0

5. Non-functional Requirements

Attribute	Requirement
Usability	Numbered menu with clear headings; every report has a titled, separated output block. Prompts tell the user exactly what to enter.
Performance	Every report is computed in memory with vectorised pandas operations and returns instantly for the data set size used.
Reliability	Invalid or non-numeric menu input never crashes the program; an error message is shown and the menu is displayed again. Out-of-range filter choices fall back to 'All'.
Portability	Runs on any operating system with Python 3 and pandas installed. No database, network access or external files are needed, because the data is embedded.
Maintainability	Each report is a separate, single-purpose function that takes a DataFrame and prints its result, so features can be added or changed independently.
Readability	Meaningful function and variable names, section comments and consistent formatting throughout the single source file.
Extensibility	The data source can be swapped for a CSV file, Excel sheet or database by changing only the loading code, since all analysis functions accept any DataFrame with the same columns.
Accuracy	All totals are computed with pandas aggregation and cross-checked by hand in the testing phase (Section 11).

6. System Architecture

The application follows a simple four-layer architecture inside a single Python module. The **data layer** holds the sales records as a pandas DataFrame. The **analysis layer** contains one function per report. The **control layer** is the `main()` menu loop that reads user choices, keeps the currently filtered DataFrame and dispatches to the right function. The **presentation layer** is the console output produced with `print()`.



Libraries: pandas (data + aggregation) | io.StringIO (in-memory CSV)

Figure 1: Layered system architecture

Data Flow

Data is loaded once at start-up into `df`. The menu loop keeps a working copy, `filtered_df`. Reports 1 to 8 always receive `filtered_df`, so a filter automatically affects every report. Choosing option 9 rebuilds `filtered_df` from the original `df`, and option 10 restores it to a full copy of `df`.

Dataset Description

Column	Type	Description
Order_ID	Integer	Unique order identifier (1001 to 1010)
Order_Date	Datetime	Date the order was placed (January to April 2026)
Customer_ID	Text	Customer identifier (C001 to C010)
Product	Text	Name of the product sold
Category	Text	Electronics, Furniture or Fashion
City	Text	Lucknow, Kanpur, Agra or Varanasi
Sales	Integer	Order value in rupees
Profit	Integer	Profit earned on the order in rupees
Payment_Method	Text	UPI, Credit Card or Cash

7. Design Diagrams

7.1 Use Case Diagram

There is a single actor, the user (analyst). Every menu option is a use case. Reports depend on the active filter state, which is controlled by the Filter Data and Reset Filters use cases.

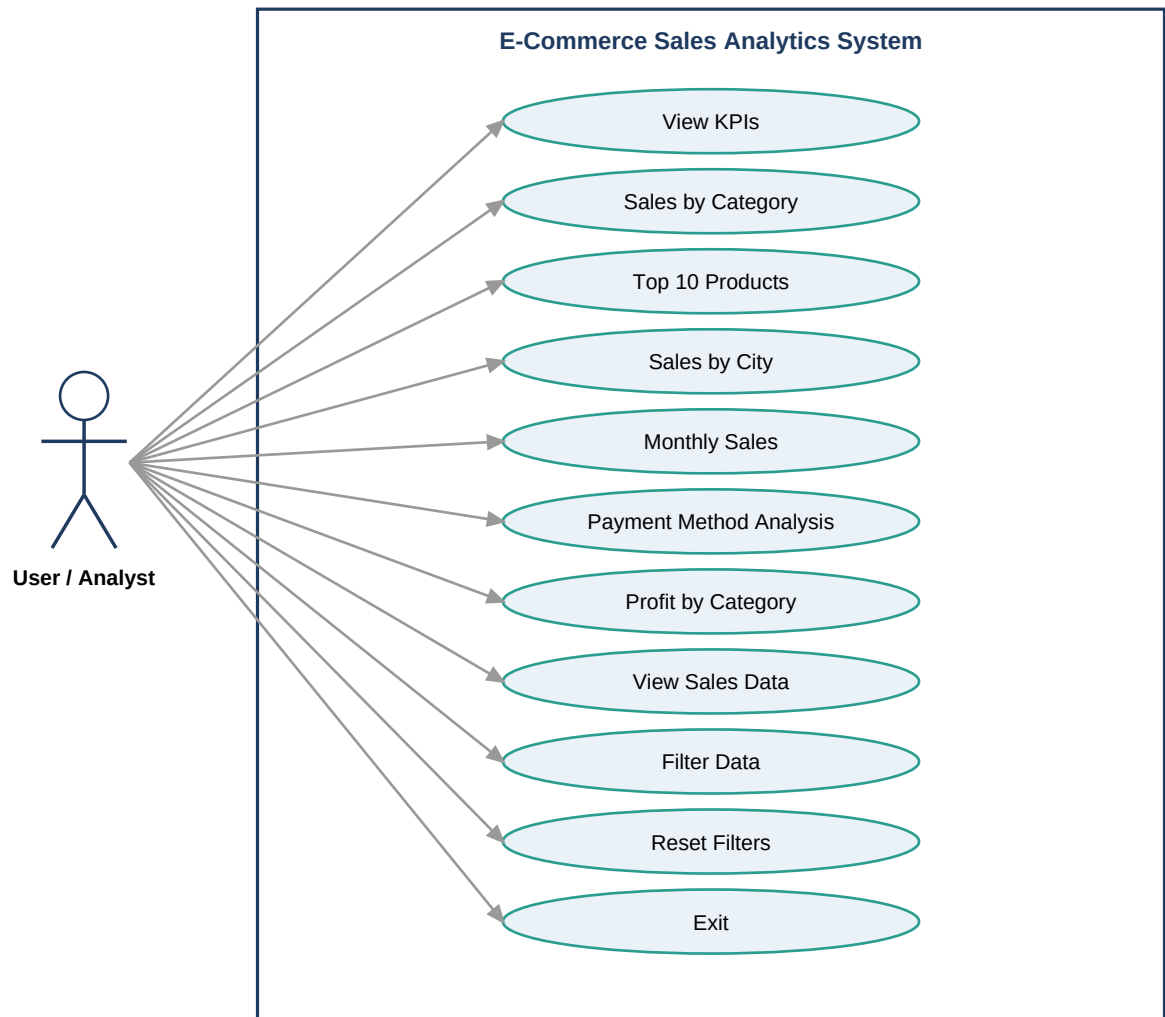


Figure 2: Use case diagram

7.2 Workflow Diagram

The flowchart shows the life cycle of one program run: data loading, the menu loop, validation, dispatch to a report and the exit path.

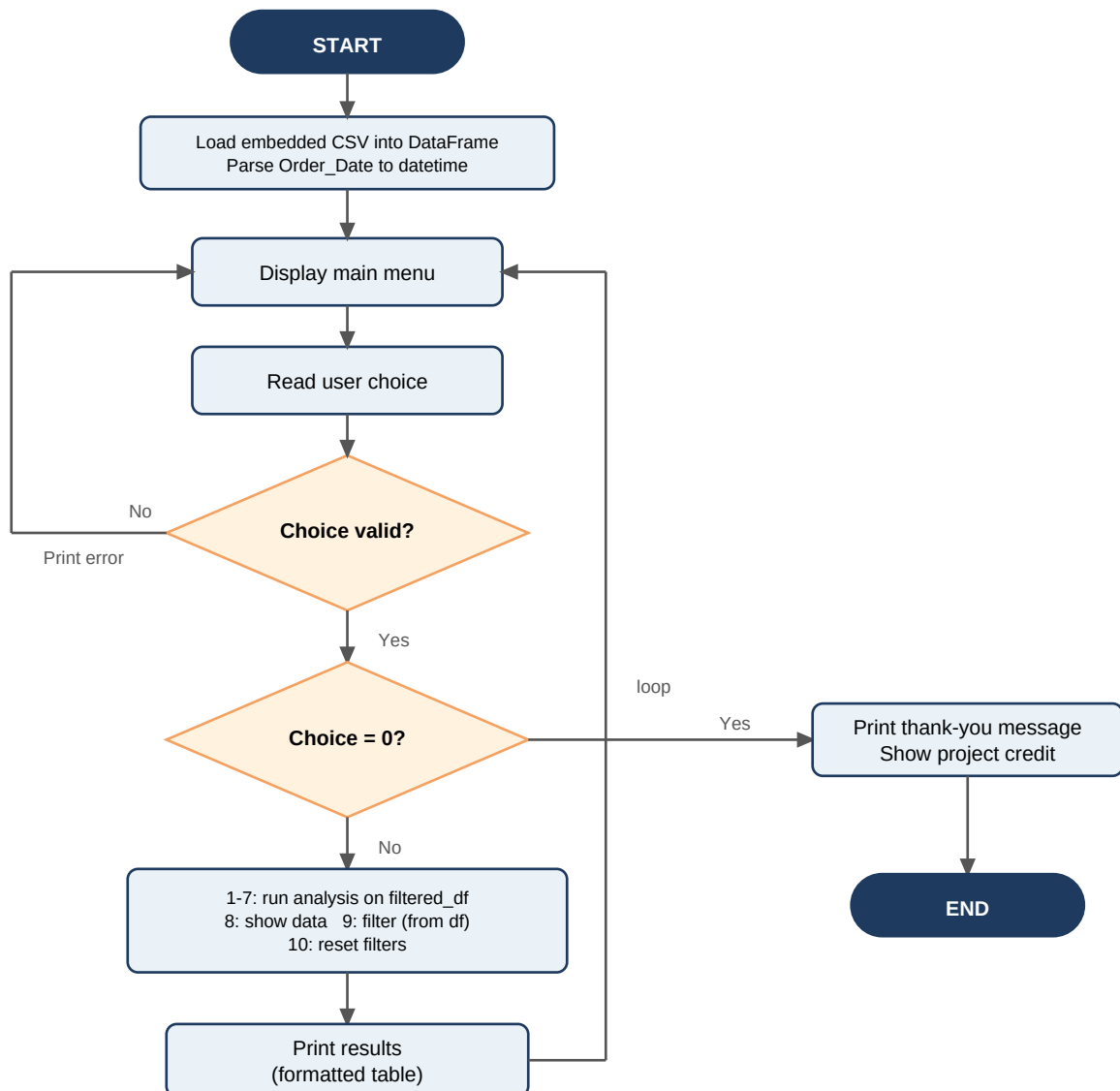


Figure 3: Workflow (flowchart) diagram

7.3 Sequence Diagram

The sequence below shows the most interactive scenario: filtering the data by category and city. Reports then run on the returned DataFrame.

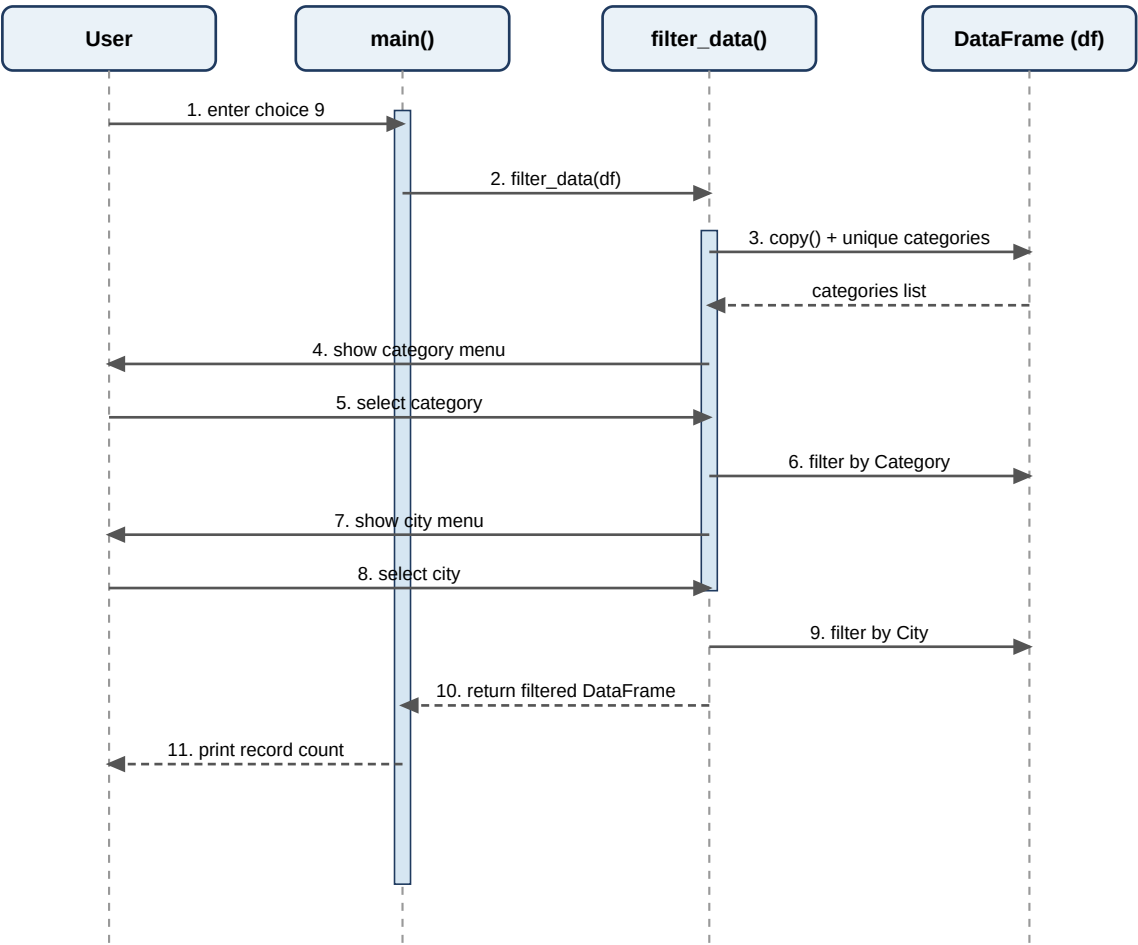


Figure 4: Sequence diagram for Filter Data

7.4 Class / Component Diagram

The project follows a procedural style, so it has no user-defined classes. The design is therefore shown as a component diagram: `main()` controls independent function components that all read the same `DataFrame`.

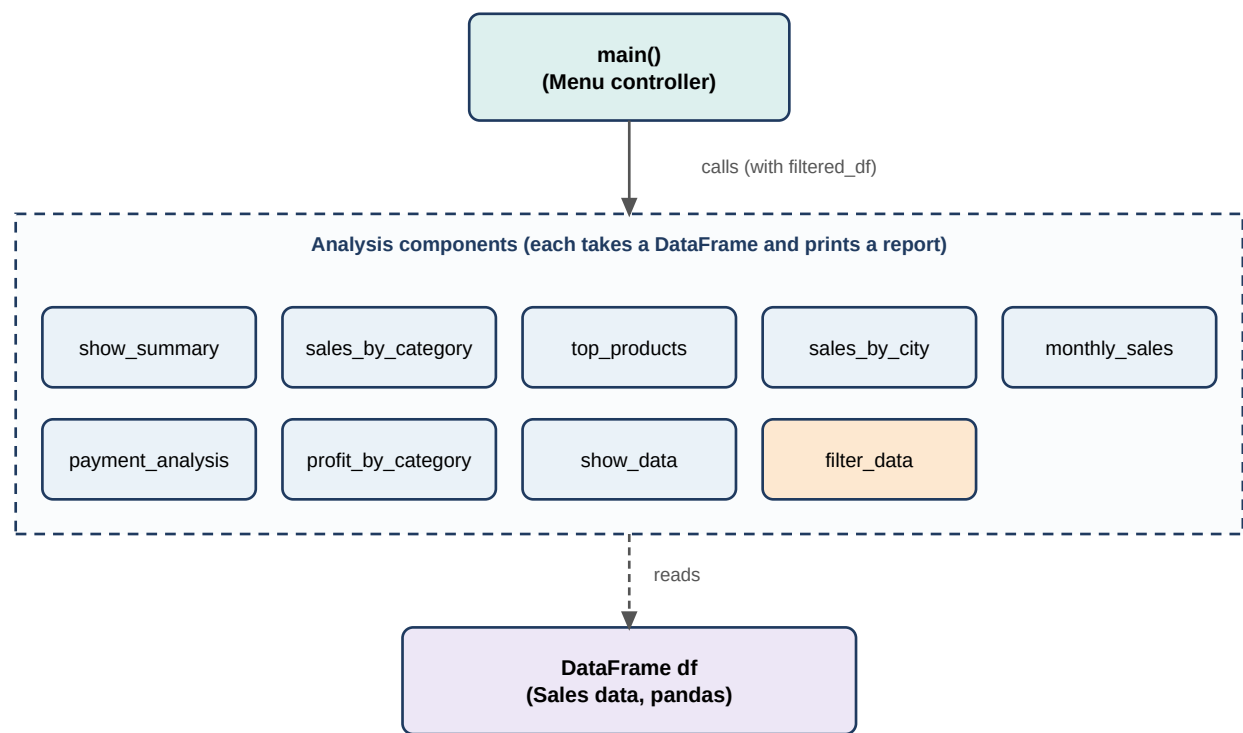


Figure 5: Component diagram

7.5 ER Diagram

The application does not use a database, so no physical ER model exists. The diagram below is a **logical** ER model that shows how the flat order table could be normalised if the project is later connected to a database (see Section 14).

Logical ER model (no database is used; data is one flat DataFrame)

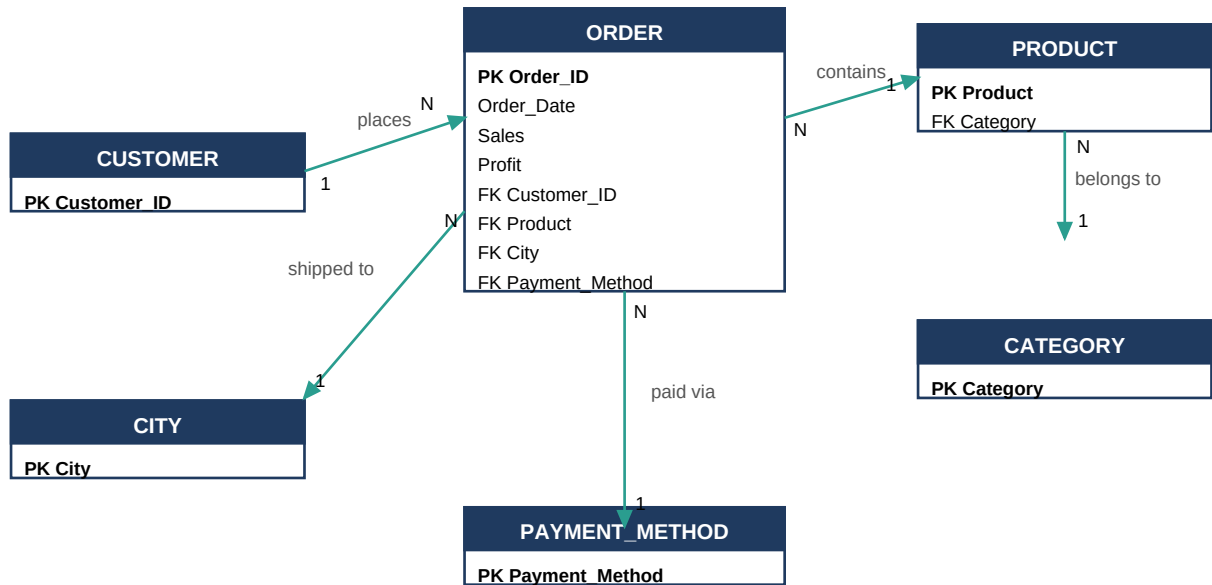


Figure 6: Logical ER diagram

8. Design Decisions and Rationale

Decision	Rationale
Use pandas DataFrame as the core data structure	groupby, sum, sort_values and Boolean filtering express each report in one or two readable lines, which is far simpler and less error-prone than manual loops over lists or dictionaries.
Embed the data as a CSV string read through StringIO	Makes the program fully self-contained: it runs anywhere with no data file to lose or path to configure, which suits a course submission and easy grading.
Console menu instead of a GUI or web app	Keeps the focus on data analysis logic, needs no extra libraries and is simple to test and demonstrate.
One function per report, each taking a DataFrame as argument	Functions do not depend on global data, so the same function works on the full or filtered data. This is what makes the filter apply to every report with no extra code.
Keep df (original) and filtered_df (working copy) separate	The original data is never modified, so 'Reset Filters' is a single copy operation and filters can never corrupt the source data.
Filter menus built from the data itself	Categories and cities are read with unique() and sorted, so new values in the data appear in the menu automatically.
Convert Order_Date with pd.to_datetime	Enables month-wise grouping through to_period('M') and correct date handling.
Use isdigit() and range checks on input	Non-numeric or out-of-range filter input quietly means 'All' and never causes an exception.
Formatted output with fixed-width separators and thousands separators	Makes the results easy to read and screenshot; currency values are shown with the rupee sign and two decimals.

9. Implementation Details

The whole application is implemented in a single file, `sales_analytics.py`, organised into clearly commented sections: data, functions, main program and project credit.

9.1 Data preparation

The CSV string is parsed with `pd.read_csv(StringIO(data))` and the date column is converted with `pd.to_datetime`. A quick print of the converted dates is included at start-up as a sanity check that parsing succeeded.

9.2 Analysis functions

Function	Logic (pandas)	Output
<code>show_summary</code>	<code>sum()</code> of Sales and Profit; <code>nunique()</code> of Order_ID and Customer_ID	Four KPIs
<code>sales_by_category</code>	<code>groupby('Category')['Sales'].sum()</code> , sort descending	Sales per category
<code>top_products</code>	<code>groupby('Product')['Sales'].sum()</code> , sort descending, <code>head(10)</code>	Top 10 products
<code>sales_by_city</code>	<code>groupby('City')['Sales'].sum()</code> , sort descending	Sales per city
<code>monthly_sales</code>	<code>groupby(Order_Date.dt.to_period('M'))['Sales'].sum()</code>	Sales per month
<code>payment_analysis</code>	<code>groupby('Payment_Method')['Sales'].sum()</code> , sort descending	Sales per payment method
<code>profit_by_category</code>	<code>groupby('Category')['Profit'].sum()</code> , sort descending	Profit per category
<code>show_data</code>	<code>to_string(index=False)</code>	Full record table
<code>filter_data</code>	<code>unique()</code> , <code>sorted()</code> , Boolean mask on Category and City	Filtered DataFrame

9.3 Filtering

`filter_data` copies the data, lists the sorted unique categories with numbers and asks the user to pick one. If the entry is a valid number, the DataFrame is reduced with a Boolean mask, for example `filtered_data[filtered_data['Category'] == selected_category]`. The same steps are then repeated for cities. Choosing 0 (or any invalid number) keeps all values for that field. The function returns the resulting DataFrame to `main()`.

```
def filter_data(data):
    filtered_data = data.copy()
    categories = sorted(data["Category"].dropna().unique())
    ...
    choice = input("\nSelect category: ")
    if choice.isdigit():
        choice = int(choice)
        if 1 <= choice <= len(categories):
            selected_category = categories[choice - 1]
            filtered_data = filtered_data[filtered_data["Category"] == selected_category]
    ... # same pattern for City
    return filtered_data
```

9.4 Main menu loop

`main()` prints the menu inside a `while True` loop, reads the choice as text and uses an `if / elif` chain to call the right function with `filtered_df`. Choice 9 assigns the result of

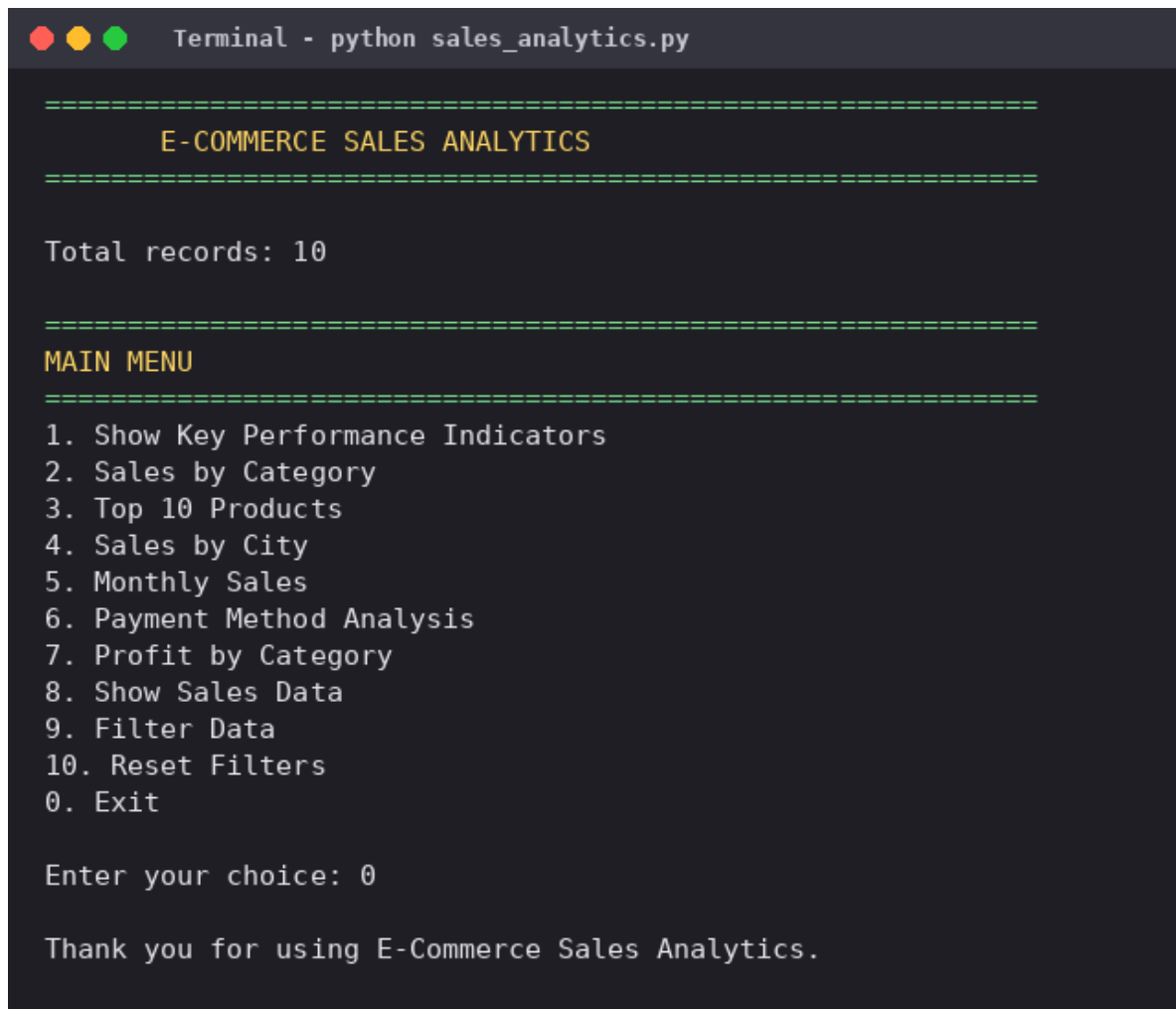
`filter_data(df)` to `filtered_df`; choice 10 assigns `df.copy()`; choice 0 breaks the loop; anything else prints an error. The program is started through the standard `if __name__ == "__main__"` guard, and a project credit block is printed at the end.

9.5 How to run

- Install Python 3 and pandas: `pip install pandas`.
- Run the program: `python sales_analytics.py`.
- Enter a menu number and press Enter; enter 0 to exit.

10. Screenshots and Results

The screenshots below were captured from actual runs of the program on the built-in data set.



```
Terminal - python sales_analytics.py

=====
E-COMMERCE SALES ANALYTICS
=====

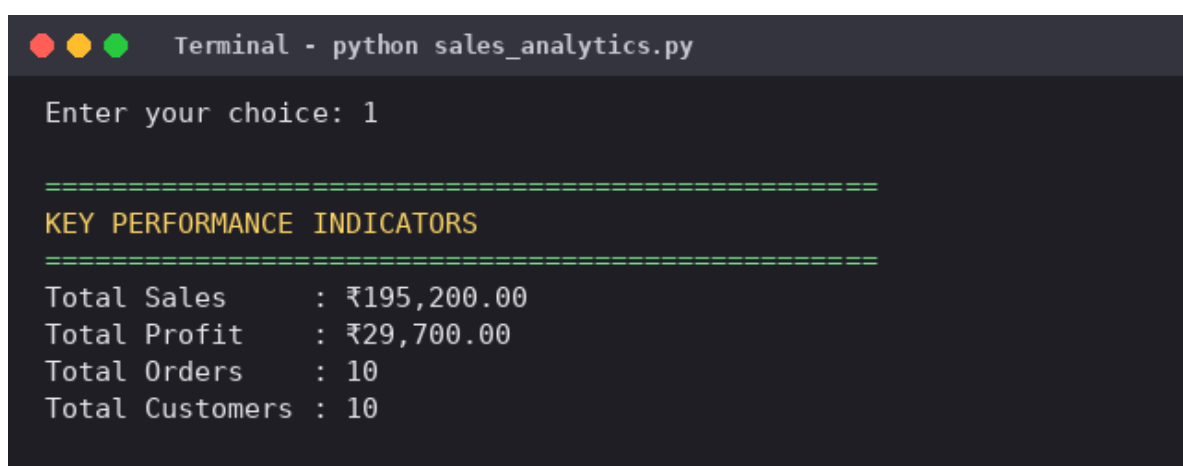
Total records: 10

=====
MAIN MENU
=====
1. Show Key Performance Indicators
2. Sales by Category
3. Top 10 Products
4. Sales by City
5. Monthly Sales
6. Payment Method Analysis
7. Profit by Category
8. Show Sales Data
9. Filter Data
10. Reset Filters
0. Exit

Enter your choice: 0

Thank you for using E-Commerce Sales Analytics.
```

Figure 7: Start-up screen and main menu



```
Terminal - python sales_analytics.py

Enter your choice: 1

=====
KEY PERFORMANCE INDICATORS
=====

Total Sales      : ₹195,200.00
Total Profit     : ₹29,700.00
Total Orders     : 10
Total Customers  : 10
```

Figure 8: Key Performance Indicators (option 1)

```
Terminal - python sales_analytics.py

Enter your choice: 2

=====
SALES BY CATEGORY
=====

Category
Electronics      163000
Furniture        20500
Fashion          11700

Enter your choice: 7

=====
PROFIT BY CATEGORY
=====

Category
Electronics      23300
Furniture        4000
Fashion          2400
```

Figure 9: Sales by category (option 2) and profit by category (option 7)

```
Terminal - python sales_analytics.py

Enter your choice: 3

=====
TOP 10 PRODUCTS
=====

Product
Laptop           55000
Smart TV         42000
Refrigerator     38000
Mobile Phone     25000
Table            12000
Office Chair     8500
Watch            6000
Shoes            4500
Headphones       3000
T-Shirt          1200
```

Figure 10: Top 10 products (option 3)

```

Terminal - python sales_analytics.py

Enter your choice: 4

=====
SALES BY CITY
=====

City
Lucknow      69500
Kanpur       67500
Varanasi     54000
Agra         4200

Enter your choice: 5

=====
MONTHLY SALES
=====

Order_Date
2026-01      83000
2026-02      25000
2026-03      49200
2026-04      38000
Freq: M

Enter your choice: 6

=====
PAYMENT METHOD ANALYSIS
=====

Payment_Method
UPI          104500
Credit Card  81000
Cash         9700

```

Figure 11: Sales by city (4), monthly sales (5) and payment analysis (6)

```

Terminal - python sales_analytics.py

Enter your choice: 8

=====
SALES DATA
=====

Order_ID Order_Date Customer_ID Product Category City Sales Profit Payment_Method
1001 2026-01-05 C001 Laptop Electronics Lucknow 55000 7000 UPI
1002 2026-01-08 C002 Mobile Phone Electronics Kanpur 25000 4000 Credit Card
1003 2026-01-12 C003 Headphones Electronics Agra 3000 800 UPI
1004 2026-02-03 C004 Office Chair Furniture Lucknow 8500 1500 Cash
1005 2026-02-10 C005 Table Furniture Varanasi 12000 2500 Credit Card
1006 2026-02-15 C006 Shoes Fashion Kanpur 4500 900 UPI
1007 2026-03-02 C007 T-Shirt Fashion Agra 1200 300 Cash
1008 2026-03-11 C008 Watch Fashion Lucknow 6000 1200 Credit Card
1009 2026-03-18 C009 Smart TV Electronics Varanasi 42000 6000 UPI
1010 2026-04-04 C010 Refrigerator Electronics Kanpur 38000 5500 Credit Card

```

Figure 12: Full sales data (option 8)

```
Terminal - python sales_analytics.py

Enter your choice: 9

=====
FILTER DATA
=====

Categories:
1. Electronics
2. Fashion
3. Furniture
0. All Categories

Select category: 1

Cities:
1. Agra
2. Kanpur
3. Lucknow
4. Varanasi
0. All Cities

Select city: 2

Filtered records: 2

Enter your choice: 1

=====
KEY PERFORMANCE INDICATORS
=====

Total Sales      : ₹63,000.00
Total Profit     : ₹9,500.00
Total Orders     : 2
Total Customers  : 2

Enter your choice: 8

=====
SALES DATA
=====

Order_ID Order_Date Customer_ID Product Category City Sales Profit Payment_Method
1002 2026-01-08 C002 Mobile Phone Electronics Kanpur 25000 4000 Credit Card
1010 2026-04-04 C010 Refrigerator Electronics Kanpur 38000 5500 Credit Card
```

Figure 13: Filtering by Electronics and Kanpur, then KPIs and data on the filtered set

```
Terminal - python sales_analytics.py

Enter your choice: 10

Filters reset successfully.

Enter your choice: 1

=====
KEY PERFORMANCE INDICATORS
=====
Total Sales      : ₹195,200.00
Total Profit     : ₹29,700.00
Total Orders     : 10
Total Customers  : 10

Enter your choice: abc

Invalid choice. Please try again.

Enter your choice: 0

Thank you for using E-Commerce Sales Analytics.
```

Figure 14: Reset filters, invalid input handling and exit

Key Findings

- Total sales are Rs. 1,95,200 (195,200) with a total profit of Rs. 29,700 from 10 orders by 10 customers, an overall profit margin of about 15.2%.
- Electronics dominates: Rs. 163,000 of sales (about 83.5%) and Rs. 23,300 of profit. Furniture follows with Rs. 20,500 and Fashion with Rs. 11,700.
- The Laptop (Rs. 55,000) and Smart TV (Rs. 42,000) are the top two products.
- Lucknow (Rs. 69,500) and Kanpur (Rs. 67,500) lead city sales; Agra is the smallest market at Rs. 4,200.
- January was the strongest month (Rs. 83,000) and February the weakest (Rs. 25,000).
- UPI is the most used payment method by value (Rs. 104,500), ahead of Credit Card (Rs. 81,000) and Cash (Rs. 9,700).

Summary of Verified Results

Report	Result (Rs.)
KPIs	Sales 195,200; Profit 29,700; Orders 10; Customers 10
Sales by category	Electronics 163,000; Furniture 20,500; Fashion 11,700
Profit by category	Electronics 23,300; Furniture 4,000; Fashion 2,400
Sales by city	Lucknow 69,500; Kanpur 67,500; Varanasi 54,000; Agra 4,200
Monthly sales	Jan 83,000; Feb 25,000; Mar 49,200; Apr 38,000
Payment analysis	UPI 104,500; Credit Card 81,000; Cash 9,700
Filter: Electronics + Kanpur	2 records; Sales 63,000; Profit 9,500

11. Testing Approach

The application was tested manually by running it end to end with scripted inputs, and every numeric result was verified independently by adding up the source records by hand. Testing covered normal use, filtering, boundary cases and invalid input.

11.1 Test strategy

- **Functional testing:** each menu option was run and its output compared with a hand-calculated expected value.
- **Data validation:** order dates were printed at start-up to confirm they were converted to datetime.
- **Filter testing:** category and city filters were combined and the KPIs re-checked against manual totals.
- **Negative testing:** non-numeric and out-of-range inputs were entered at the menu and filter prompts.
- **Cross-check:** category, city and payment totals each add up to the overall Rs. 195,200 sales figure.

11.2 Test cases

ID	Test	Input	Expected result	Result
T1	KPIs on full data	1	Sales 195,200; Profit 29,700; Orders 10; Customers 10	Pass
T2	Sales by category	2	Electronics 163,000 > Furniture 20,500 > Fashion 11,700	Pass
T3	Top products	3	Laptop 55,000 first; Smart TV 42,000 second; 10 rows	Pass
T4	Sales by city	4	Lucknow 69,500; Kanpur 67,500; Varanasi 54,000; Agra 4,200	Pass
T5	Monthly sales	5	Jan 83,000; Feb 25,000; Mar 49,200; Apr 38,000	Pass
T6	Payment analysis	6	UPI 104,500; Credit Card 81,000; Cash 9,700	Pass
T7	Profit by category	7	Electronics 23,300; Furniture 4,000; Fashion 2,400	Pass
T8	Show data	8	All 10 rows with 9 columns	Pass
T9	Filter category + city	9, 1, 2	2 records (Electronics in Kanpur)	Pass
T10	KPIs after filter	1	Sales 63,000; Profit 9,500; Orders 2; Customers 2	Pass
T11	Reset filters	10, then 1	Full totals restored (195,200 / 29,700)	Pass
T12	Invalid menu choice	abc	'Invalid choice' message, menu shown again	Pass
T13	Filter with 0	9, 0, 0	All 10 records retained	Pass
T14	Filter out-of-range	9, 99, 99	Treated as 'All'; no crash	Pass
T15	Exit	0	Thank-you message, loop ends, credit printed	Pass

12. Challenges Faced

Challenge	How it was handled
Making one filter work for all reports	Reports were written as functions that take a DataFrame argument instead of using the global data. The menu passes the filtered DataFrame, so no report needed changes.
Handling invalid user input without crashes	Menu choices are read as text and compared as strings. Filter input is checked with <code>isdigit()</code> and a range test before converting to int.
Monthly grouping	Dates were converted with <code>pd.to_datetime</code> and grouped with <code>dt.to_period('M')</code> to get clean YYYY-MM labels.
Readable console output	Fixed-width separator lines, headings and <code>to_string()</code> output were used so the tables align and can be captured in screenshots.
Filter behaviour across repeated use	<code>filter_data</code> always starts from the original df. This keeps each filter independent and predictable, and Reset Filters remains a simple operation.
Showing currency correctly	f-string formatting with thousands separators and two decimals (for example, <code>{value:,.2f}</code>) was used for the rupee amounts.
Keeping the project self-contained	The sample data was embedded as a CSV string, so the program has no file-path or environment issues when run by someone else.

13. Learnings and Key Takeaways

- **pandas fundamentals:** reading data, converting types, groupby and aggregation, sorting, ranking with head(), unique values and Boolean filtering.
- **Function-based design:** small, single-purpose functions with parameters are easy to test, reuse and extend.
- **Separating original and working data:** keeping df untouched and using a filtered copy prevents accidental data loss and simplifies reset.
- **Input validation:** user input must never be trusted; simple checks make a program robust.
- **Turning data into insight:** the same table can answer many business questions when grouped in the right way. Here it revealed a strong dependence on Electronics and on two cities.
- **Documentation and testing discipline:** verifying outputs by hand and documenting design choices makes the project easier to explain and maintain.

14. Future Enhancements

Enhancement	Benefit
Load data from CSV, Excel or a database (SQLite, MySQL)	Work with real, larger data sets; enables the normalised ER design in Section 7.5.
Charts with matplotlib or seaborn	Visual bar, line and pie charts for category, city and monthly trends.
Export reports to CSV, Excel or PDF	Share results without copying from the console.
Date-range and payment-method filters, and filter stacking	More flexible analysis; combine several filters without restarting.
Profit margin, average order value and growth rate KPIs	Deeper business insight beyond totals.
Customer analysis (repeat customers, top customers)	Supports retention and marketing decisions.
Graphical or web interface (Tkinter, Streamlit, Flask)	Friendlier for non-technical users.
Automated unit tests with pytest	Catch regressions when features are added.
Split the code into modules and a package	Better structure as the project grows.

15. References

- VITyarthi - *Build Your Own Project: General Project Instructions & Submission Guidelines* (course-provided document).
- CSE1021 - Problem Solving and Object-Oriented Programming: *50 Mini-Project Titles & Problem Statements* (course-provided document, used as a reference for syllabus-aligned Python concepts).